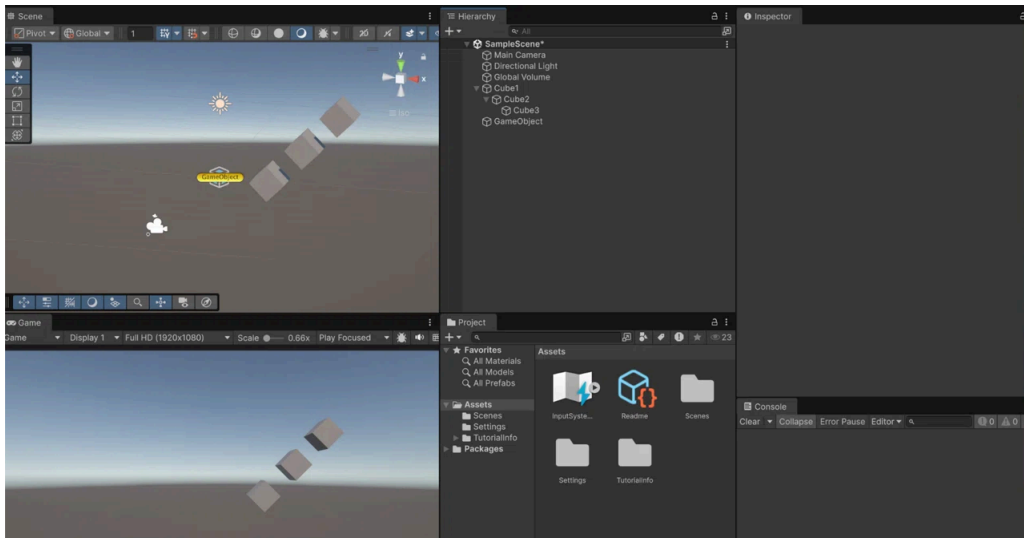




🎮 UnityEditorの各ビューを理解しよう



むぎ

2025年11月19日 01:57

...

Unityを起動すると表示される「Unity Editor（ユニティエディタ）」は、ゲームを「作る」「配置する」「確認する」ための作業スペースです。



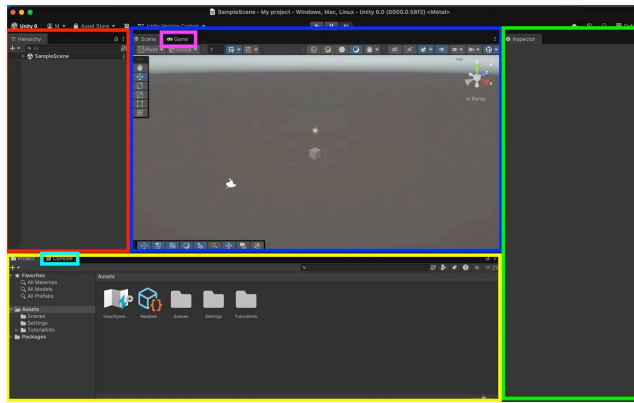
Unity Editor全体を見てみよう

【実践】

まずはUnityProjectを立ち上げ、目視しながら理解していきましょう👍

Unity Editorには、主に6つのビュー（画面）が配置されています。

- Sceneビュー(青)
- Gameビュー(ピンク)
- Hierarchyビュー(赤)
- Inspectorビュー(緑)
- Projectビュー(黄)
- Consoleビュー(水)



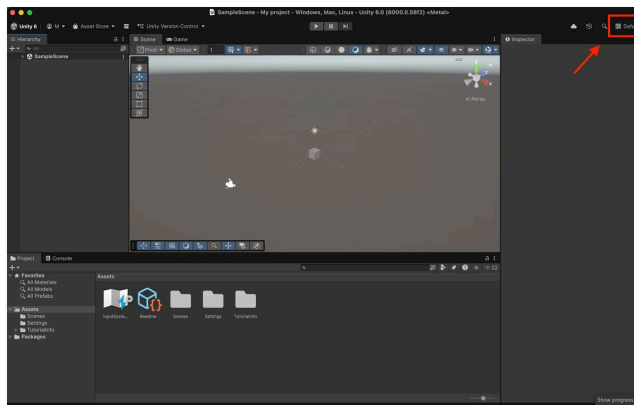
初めはなんとなく見渡すだけでOK！

👁️レイアウトを変更しよう

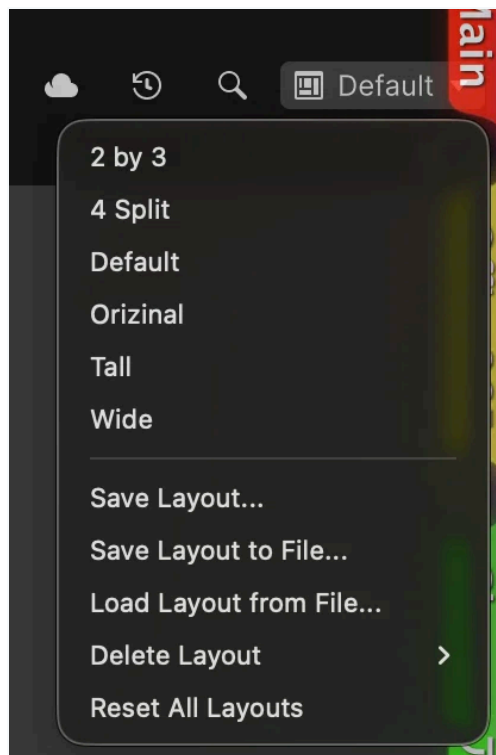
今のままだとGameビューやConsoleビューが隠れていてみずらいです。
後述しますが、この2つのビューは常に見えておいてほしいビューなので...

【実践】

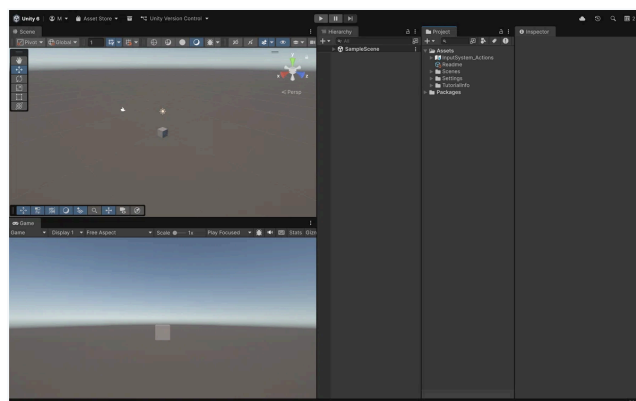
レイアウトを変更しましょう👍



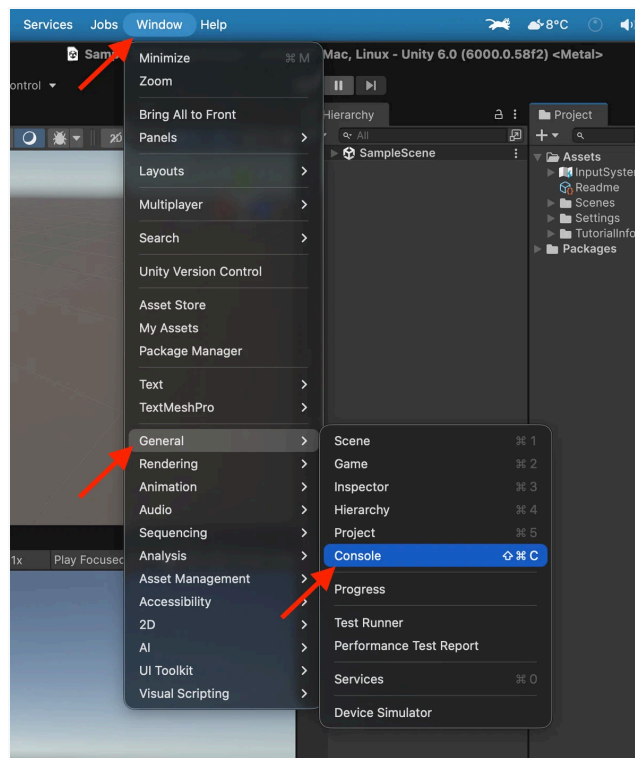
右上のボタンをクリック。



レイアウトメニューが出てきました。
今回は「2by3レイアウト」を元にカスタマイズしてみます。
「2by3」をクリック。



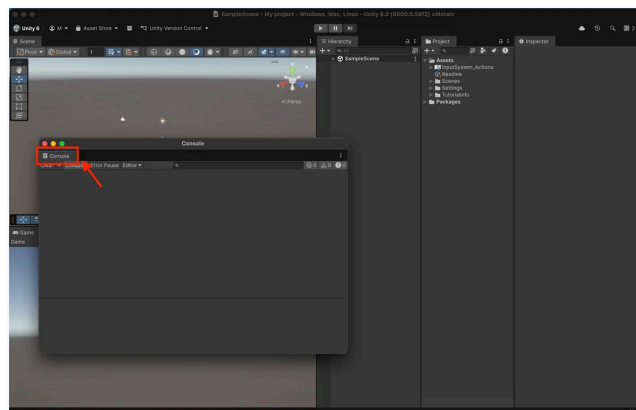
このようなレイアウトになりました。
ただこれだとConsoleビューが見えません。
Consoleビューを追加しましょう。



Windowタブ/General/ConsoleからConsoleタブを追加。

Tips

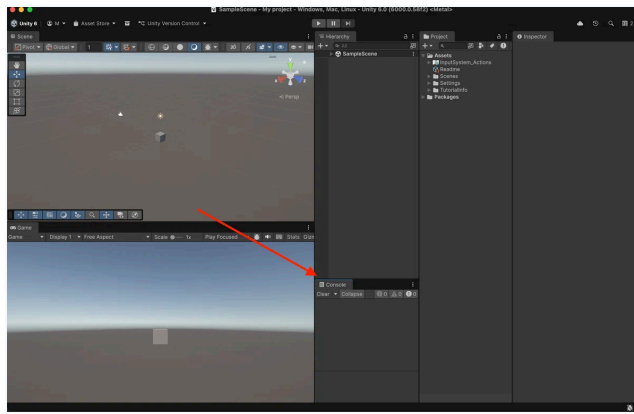
「Window/General/」ここからビューを追加できるので、間違えてビューを消したっちゃ！といった場合はここから追加しましょう👍



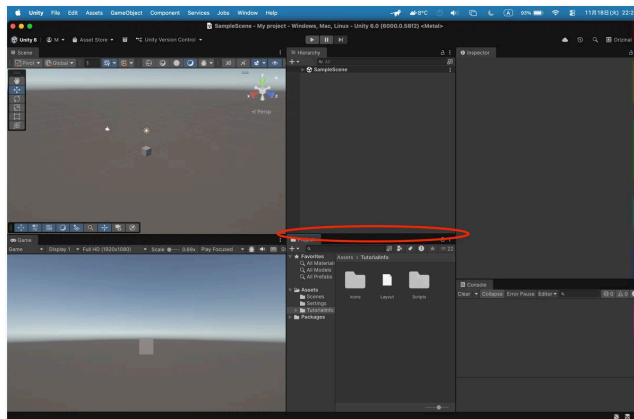
出てきたウィンドウのこの部分をD&Dで...

Tips

D&D・・・ドラッグ&ドロップ。マウスでファイルやウィンドウなどを移動すること。

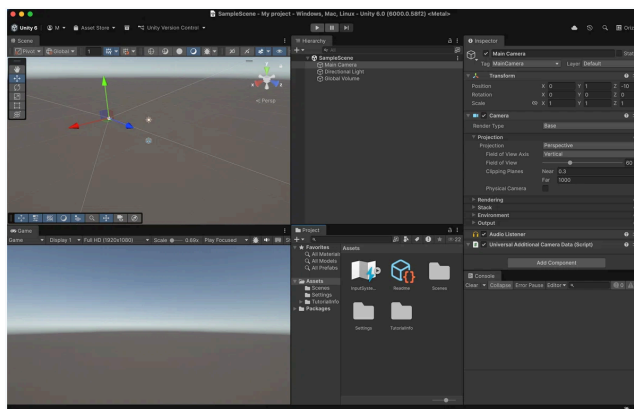


ここに持ってくる！



ビューとビューの間にカーソルを合わせると、ビューを引き伸ばすことができます。
情報量が多いビューなどは広めにとるといいねd(´´)

操作覚えてきたので、挑戦です！
このレイアウトにしてみてください👍



例

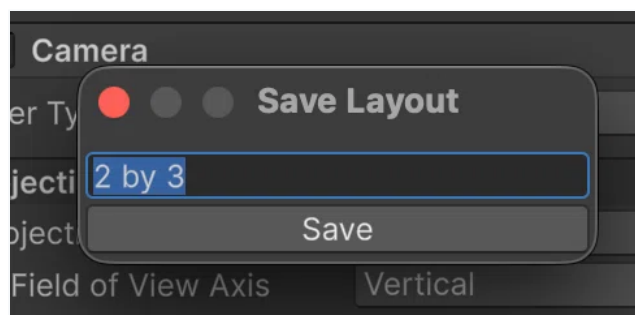
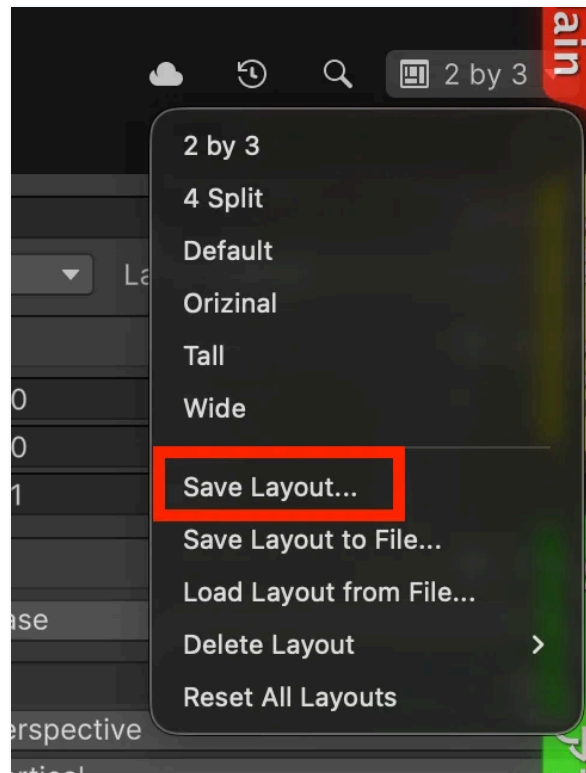
普段は私はこのレイアウトではありませんでしたが、最近はこのレイアウトもいいかも...？となっています(´▽`)/

👁️レイアウトを保存しよう

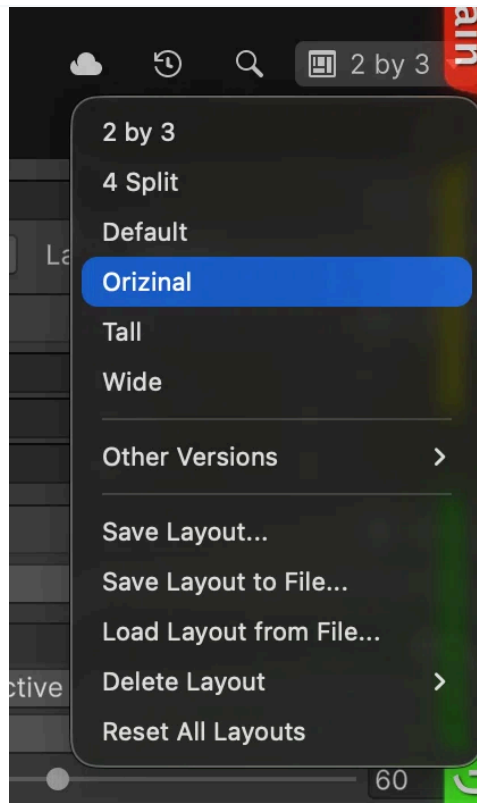
新しいUnityプロジェクトを立ち上げるごとにレイアウトを整えるのは面倒ですよね。なので、作ったレイアウトを保存しましょう👍

【実践】

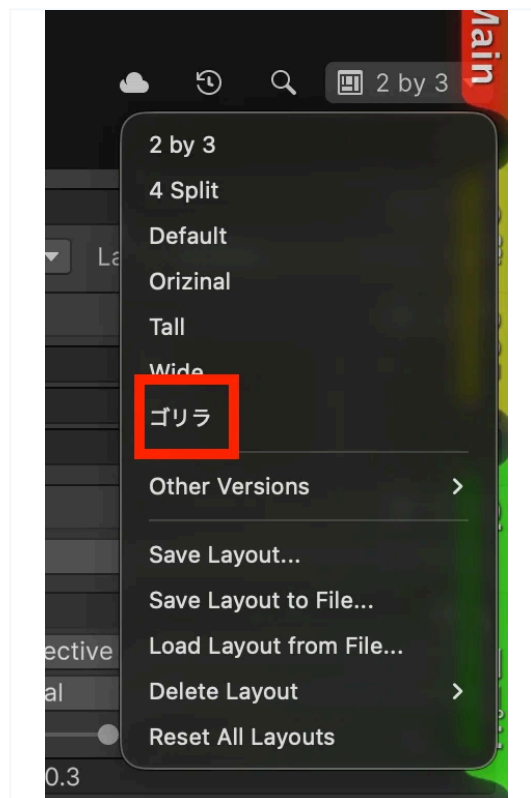
レイアウトメニューから、「Save Layout...」をクリック。
ここからレイアウトを保存できます。



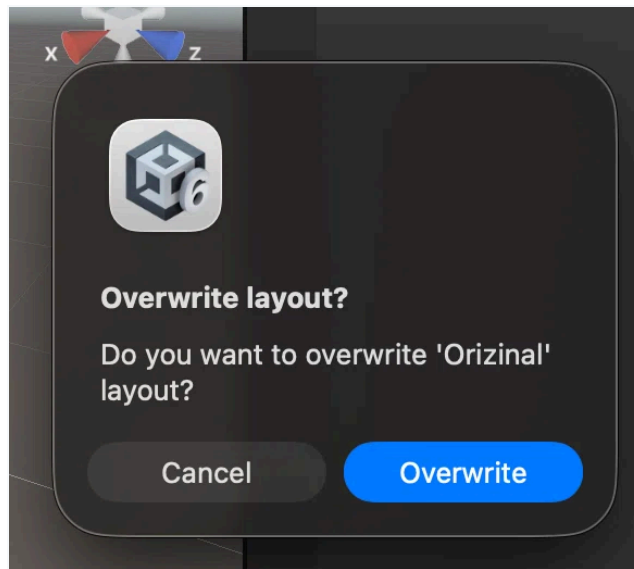
名前は好きなのにしましよう👍
私は「Orizinal」にしようかな！
名前が決まったら「Save」をクリック！



保存した「Orizinal」がテンプレートとして記録されていますね👏
これでどのプロジェクトでもここを押すだけでお気に入りのレイアウトでゲーム開発ができます！



ちな、日本語もいける🍡

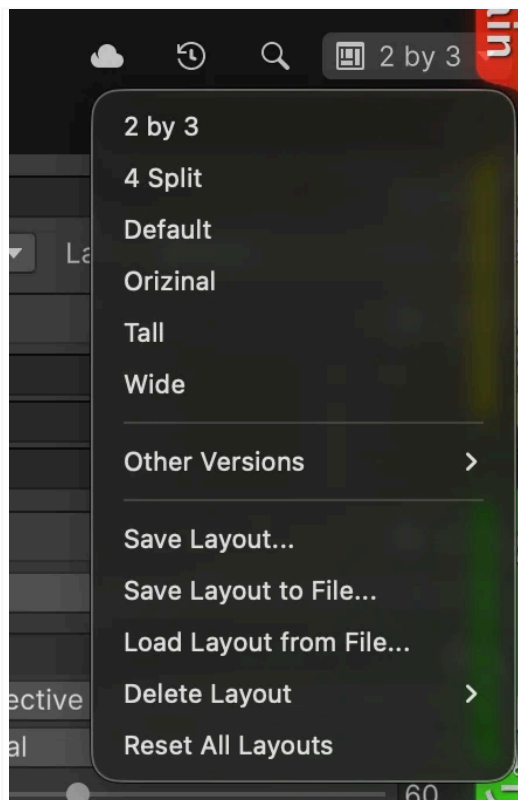


このようなダイアログが出てきた場合
保存しようとした名前と同じ名前のテンプレートレイアウトがあります。
上書きしていいなら「Overwrite」をクリック。
Overwrite・・・上書き

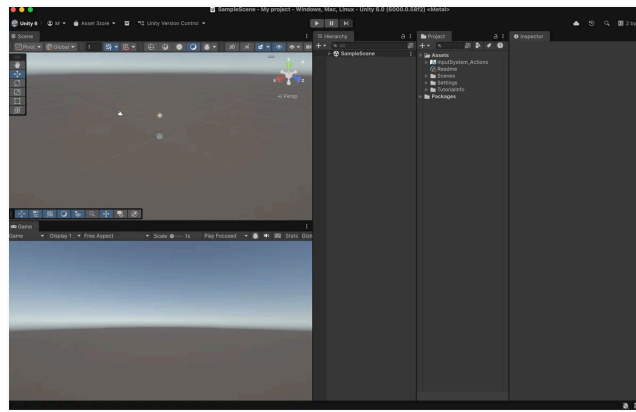
👁️ レイアウト設定の各項目の説明

【実践】

レイアウト設定、色々触って覚えていきましょう！

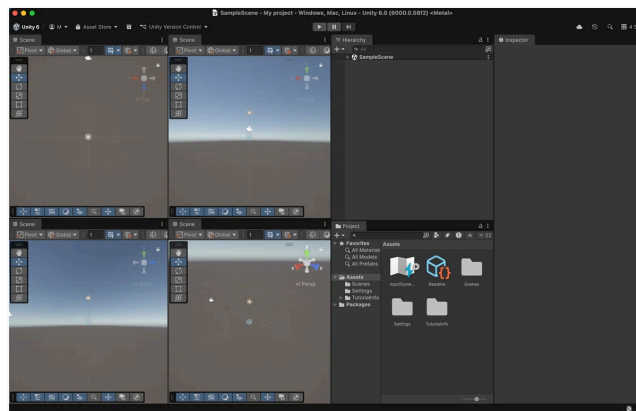


2 by 3・・・SceneビューとGameビューが上下に分かれた配置。プレビューしながら作業しやすいが、Consoleビューがないのが難点。



2by3

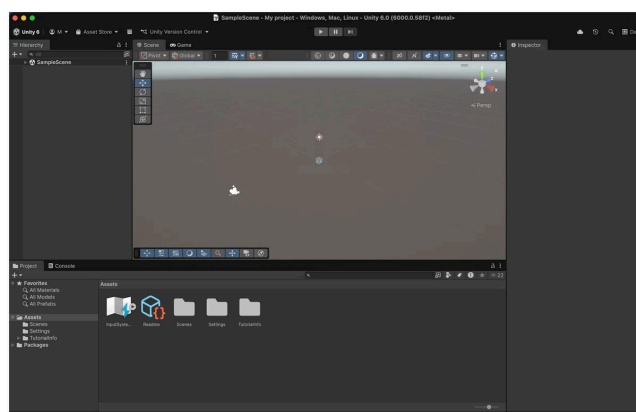
4 Split・・・4つのSceneビュー（上・前・横・パース）を同時表示。DCCツールのような作業向き



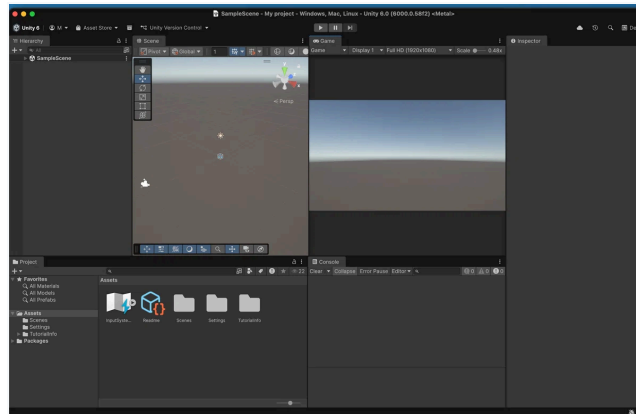
Tips

DCCツール・・・「**D**igital **C**ontent **C**reation（デジタルコンテンツクリエーション）」の略で、モデリング、シェーディング、ライティング、アニメーション、レンダリングなど、3DCG制作に必要な様々な機能が統合されたソフトウェアのこと。

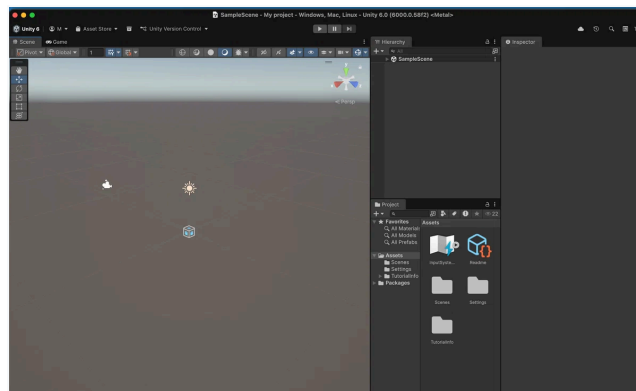
Default・・・Unityの標準レイアウト。YouTubeやUdemyなどではこれ基準にしている動画が多いのでおすすめではある。が、GameビューとConsoleビューが隠れているのが難点。



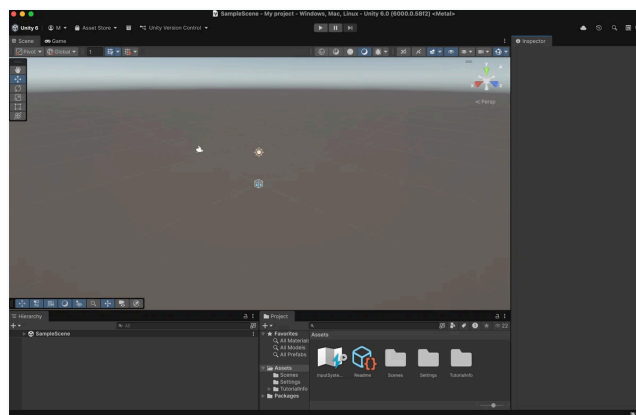
私はDefaultをカスタマイズしてこのようにレイアウトを変えてゲーム作ってたよ。結構このレイアウトもおすすめ。(Gameビューはちょっとみづらい🐼)



Tall・・・縦に長い構成。Scene/Game/Inspectorが広く見やすい。ただやはりGameやConsoleが見えないのが難点。



Wide・・・横に広い構成。Sceneビューを広く使いたいときに便利。Hierarchyビュー/Projectビューが狭いのが難点。Gameビュー/Consoleビューも見えない...



Other Versions・・・Unityの他のバージョンで保存したレイアウトを読み込む。

Save Layout...・・・現在のウィンドウ配置を名前を付けて保存する。

Save Layout to File...・・・配置を外部ファイル（.wlt）として保存。他の人に共有できるぞ！！

Load Layout from File...・・・.wltファイルからレイアウトを読み込む。

Delete Layout・・・保存済みのレイアウトを指定し、削除する。

Reset All Layouts・・・すべてのレイアウト設定を初期状態に戻す。

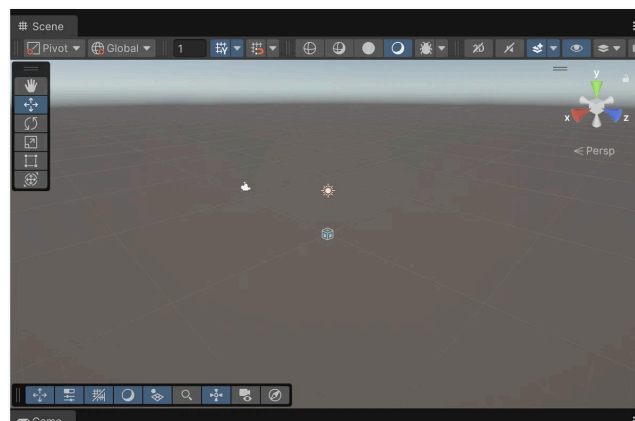
色々レイアウトについて説明しましたが、

- ベースで使うオリジナルレイアウト1つ
- 用途に合わせたレイアウトを用途ごとに切り替え

このように運用すると良いかなと思います！！

 **お待たせしました！各ビューについて詳しくみていきましょう！**

 **Sceneビュー：ゲームの「現場」**

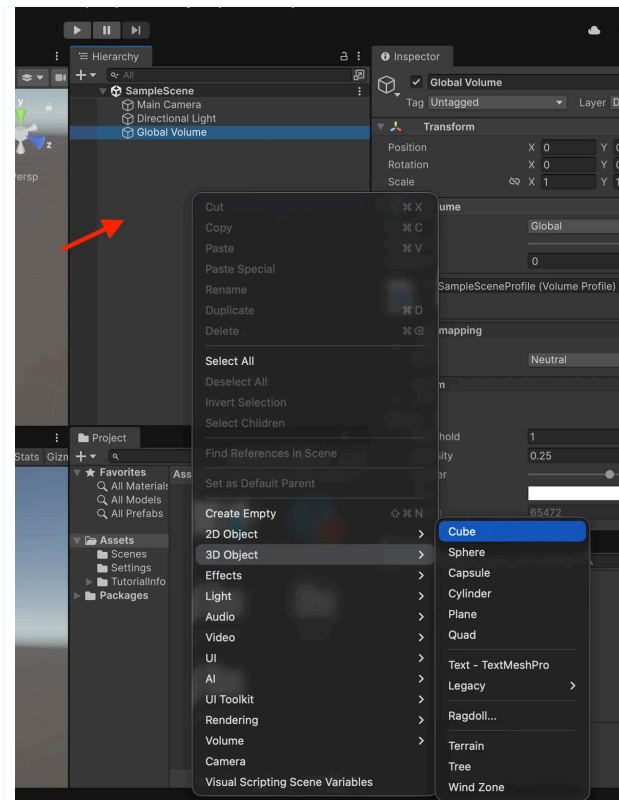


Sceneビューは、オブジェクトを**配置・移動・回転・スケール**するためのエリアです。
実際のゲーム空間を作る「作業現場」と考えると分かりやすいです。

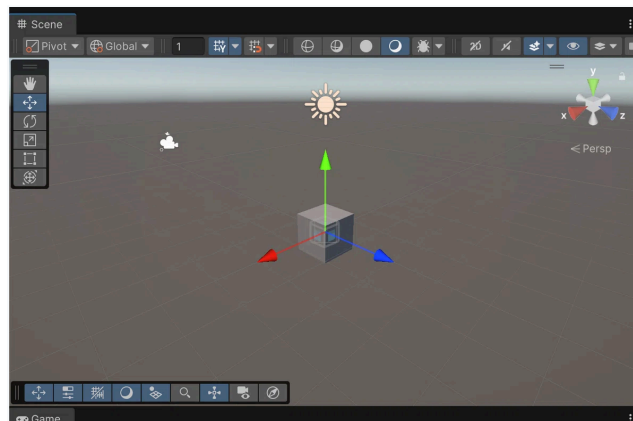
新居に引っ越したら買った家具の配置どうしようかな〜とあれこれ置き直したり角度変えたりして配置すると思います。あれです👉

【実践】

Sceneビューにキューブを置いて、動かしたりしてみましょう！



Hierarchyビューを右クリック。
3D Object/Cubeからキューブを生成。



Gizmoを使い、位置・回転・拡縮を行ってください。



名前：ショートカット：説明
 Hand Tool（ハンドツール）：Q
 カメラ視点をドラッグして移動。オブジェクトは動かさない。

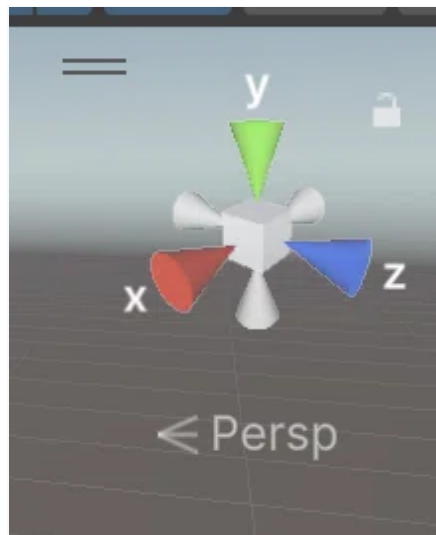
Move Tool（移動ツール）：W
 オブジェクトをX/Y/Z軸方向に移動

Rotate Tool（回転ツール）：E
 オブジェクトを回転

Scale Tool（スケールツール）：R
 オブジェクトを拡大・縮小

Rect Tool（矩形ツール）：T
 2D UIなどを矩形で位置・サイズ調整
 RectTransformのWidth/Hightを調整できる！！

Transform Tool（統合ツール）：Y
 移動・回転・スケールを一体化して操作できるモード

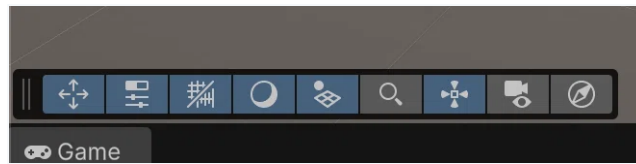


錠前マーク：解除されていたら視点を回転できる。施錠されていたら回転不可。
 X/Y/Z：Gizmo。各方位押すとフロントビューになったりサイドビューになったりするぞ。

Persp/Iso：カメラの表示モード。パースかアイソかを選択できる。

パース：**透視投影**(遠近感がある)

アイソ：**平行投影**(遠近感がない)



「表示ツールバー」(正式名称知らん)
Sceneビュー上にあるツールの表示/非表示ボタン群

【実践】

表示/非表示繰り返してどのような変化があるか確認してみよう！



「ツールバー」(正式名称知らん)
Pivot / Center：ツールの基準点切り替え。
オブジェクトの中心か基点のどちらを基準に操作するか。
ImageコンポーネントなどのUIだと分かりやすいぞ。

Global / Local：座標系の切り替え。
ワールド座標かローカル座標か。

数値入力欄 (1)：スナップ間隔。小さい値になる程細かくスナップ。
スナップ(snap)・・・カチッとハマる。吸い付くという意味。
オブジェクトをグリッド単位でピタッと配置させる。
オブジェクトを移動させてみてください。
設定した数値ごとにピタッと移動していくはず。

位置スナップ：移動にスナップを効かす

回転スナップ：回転にスナップを効かす

スケールスナップ：拡大縮小にスナップを効かす

シェーディング：シェーディングモードの切り替え
ワイヤフレーム/シェード/ライティングなし/ライティングあり...

描画デバックモード：アルベドのみなど各種設定あり。

2D：2Dモード切り替え

スピーカー：音声On/Off

✦ アイコン：スカイボックスやポストプロセス、フォグなどのOn/Off

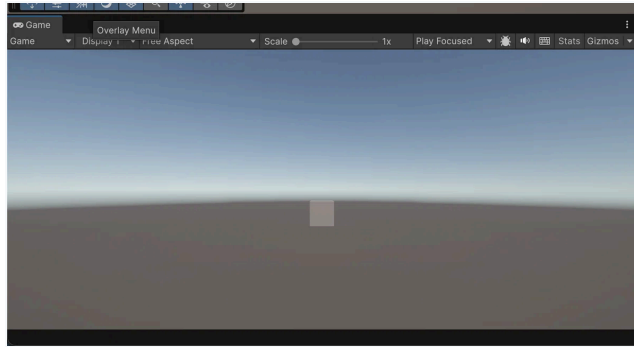
レイヤー：レイヤーごとの表示/非表示を設定

カメラ：視野角などのSceneカメラの各種設定

円形のワイヤフレームアイコン：Scene上のアイコンの表示/非表示設定など...

これらの操作をし、ゲームの「空間」をしっかり把握しましょう！

🎮 Gameビュー:ユーザーが見る画面



再生ボタン ▶ を押すと、Gameビューが**ユーザー視点の映像**を表示します。
Sceneビューは「作る側」、Gameビューは「遊ぶ側」の視点です。

📝 Tips

Gameビューでのクリックやドラッグは基本的に無効❌

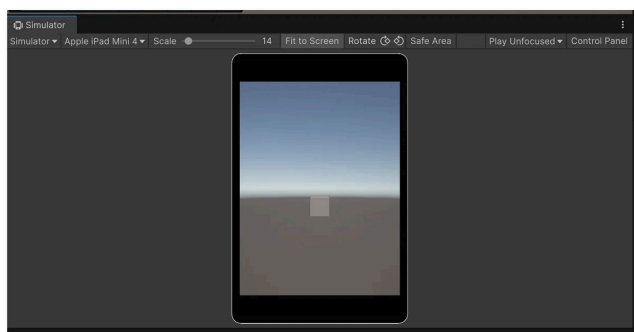


【実践】

実際に触りながら各操作をなんとなくで把握していこう！

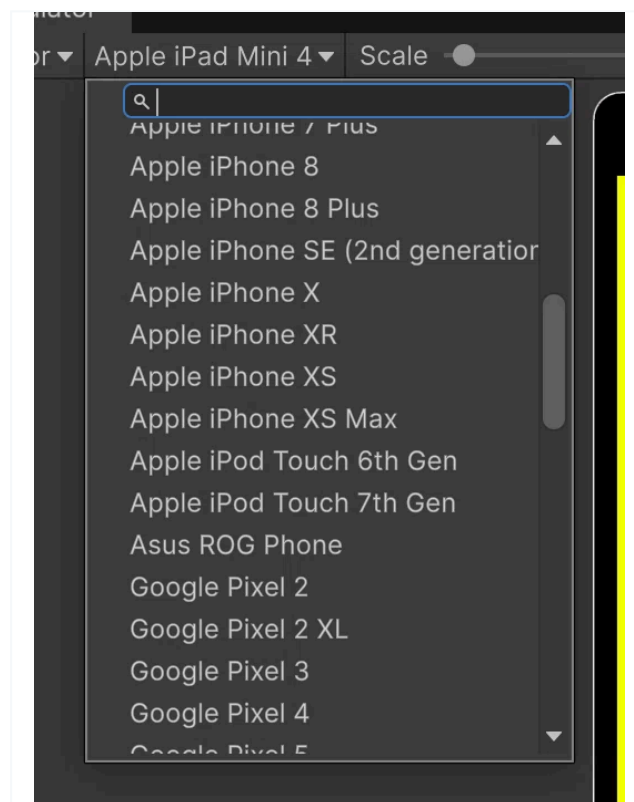
Game/Simulator

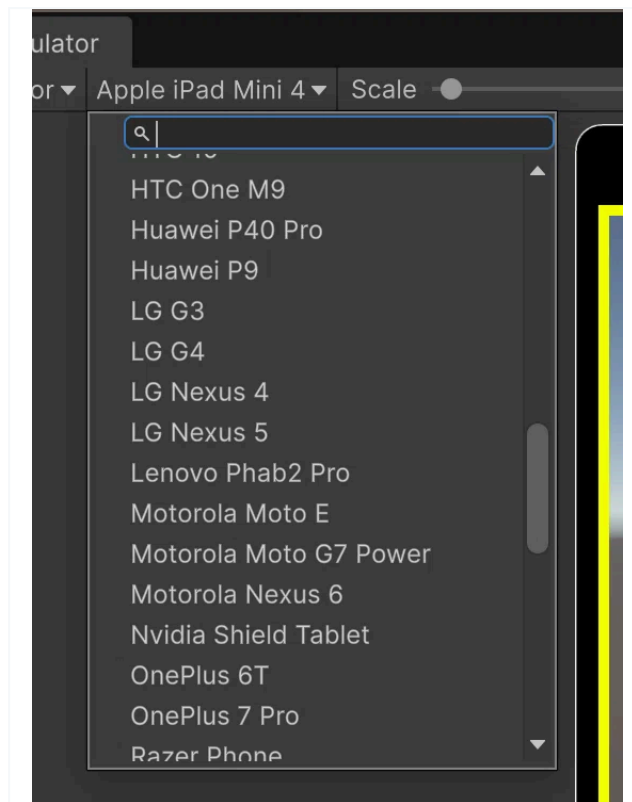
実際のゲーム画面はどう映るか



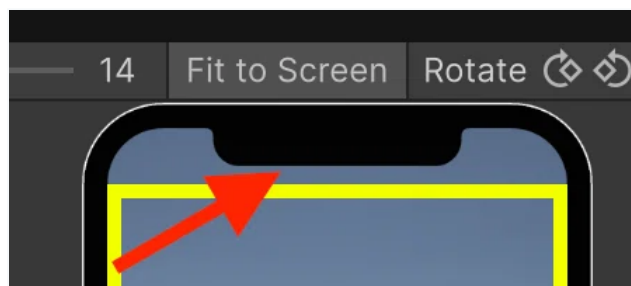
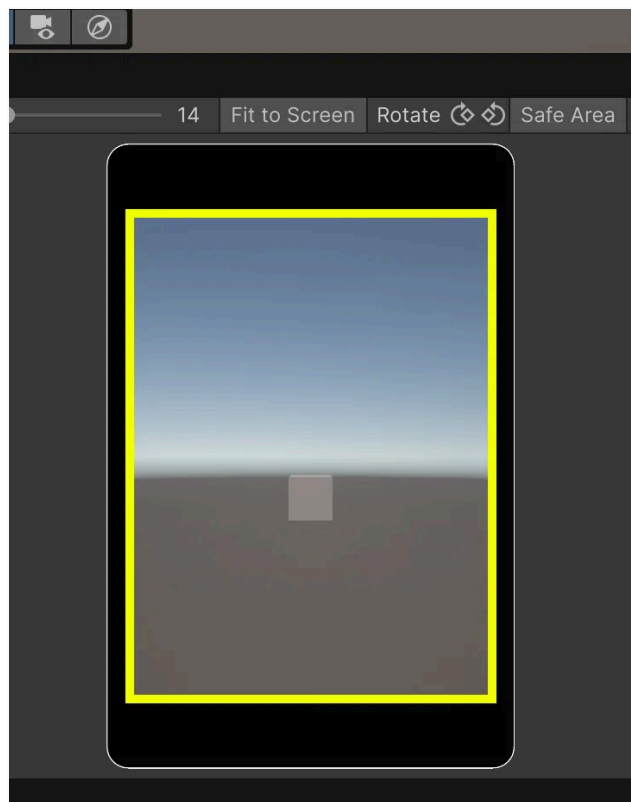
Simulatorにすることで、モバイル端末に合わせた解像度ができる👏

めっちゃある。





Safe Areaを押すことで、ユーザーが押せる/見える範囲がわかる。



ここを「ノッチ」という。

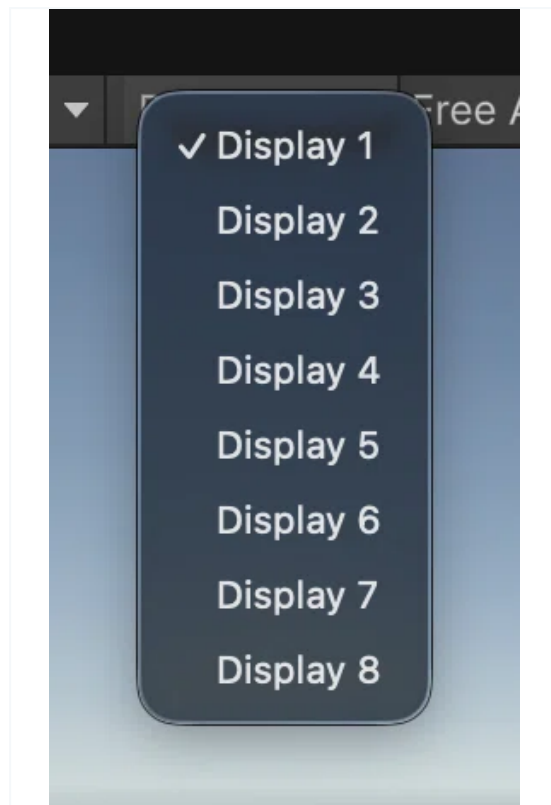
詳しくはこちら辺参考に

https://join.biglobe.ne.jp/mobile/sim/gurashi/tips_0137/

スマホゲーならSafeArea内でボタンなどのUIを入れるように⚠

Display1

ゲームを出力するディスプレイの選択。マルチディスプレイ対応する時に使ったりする。基本はDisplay1でOK。



Free Aspect

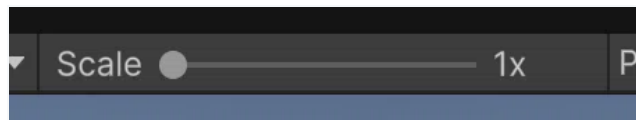
画面比率(アスペクト比)。初期設定は「Free Aspect」 9:16や1920x1080など設定できる。



「+」から自分で作ることも可能。

スケールスライダー

拡大縮小できる。

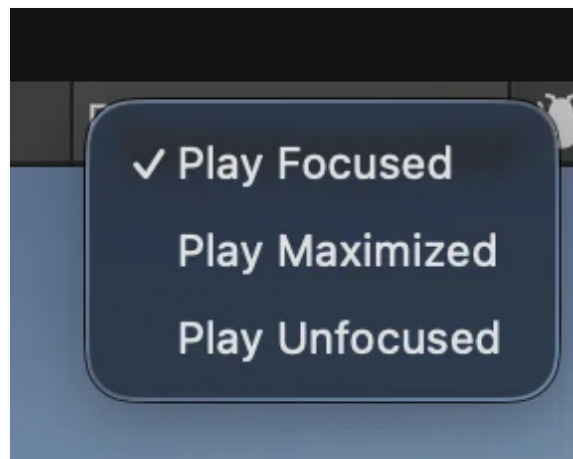


Play Focused

Play Focused・・・再生ボタンを押すと、自動的にGameビューが選択状態（入力を受け取れる状態）になる。

Play Maximized・・・再生ボタンを押すと、Gameビューが画面いっぱいに広がる。

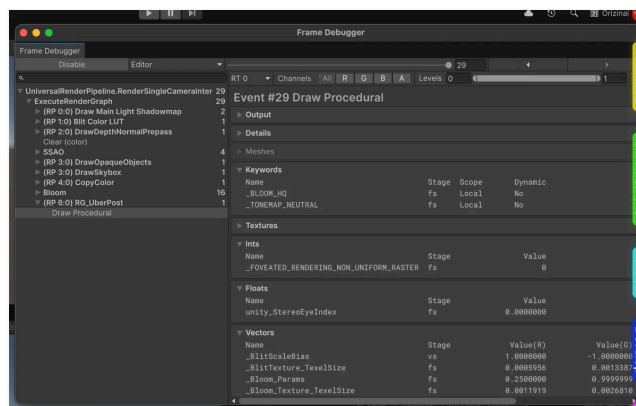
Play Unfocused・・・再生してもGameビューにフォーカスを当てない。



デバッグモード

実行中の描画などの細かいところをデバッグできる。

RenderDoc(<https://renderdoc.org/>)みたいなもの。



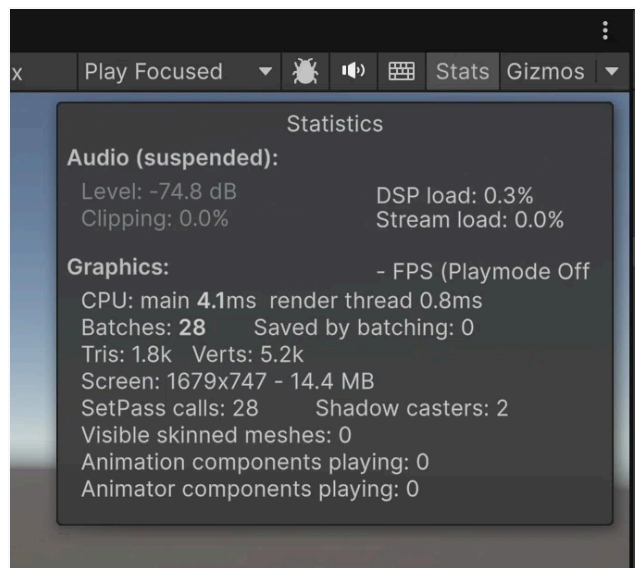
オーディオミュート

音声をミュートできる。

Stats(スタッツ)

フレームレート（FPS）、メモリ使用量、描画オブジェクト数などを表示。パフォーマンス確認に使う。

例えば、カリング処理を行った結果XX個の描画数→XX個まで減少できた。などに便利〜♪



Gizmo

ギズモをゲームビューにも表示するか

Hierarchyビュー:シーン内のオブジェクト一覧

シーンに存在するすべてのオブジェクトを階層構造で表示。

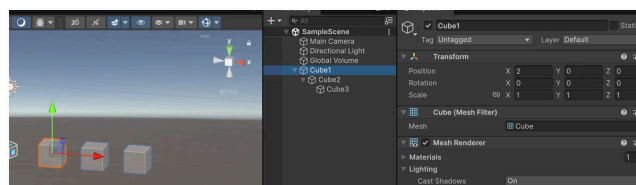
EmptyObjectを使ってグループ化することもできます。

【実践】

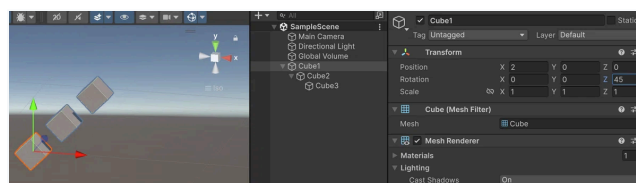
実際に触りながら各操作をなんとなくで把握していこう！



オブジェクト同士に親子関係を持たせることでグループ化したり整理させたり
位置や回転を子供に影響させたりと使い方は様々！



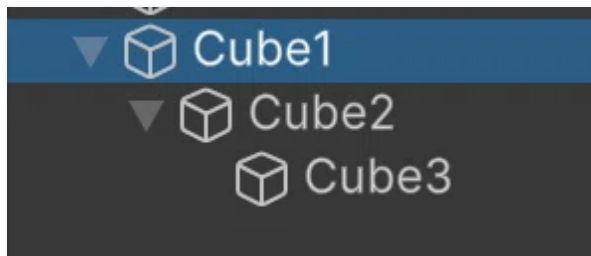
Cube3つをこのような親子関係にしてみよう！
Cube1,2,3それぞれPositionXを2に設定してみよう。
親のPositionの影響を受け、Cube3は原点からX座標に+6移動しているはず！



Cube1のRotationZ45にしてみました。
子供も影響を受けCube1を中心に回転していますね。

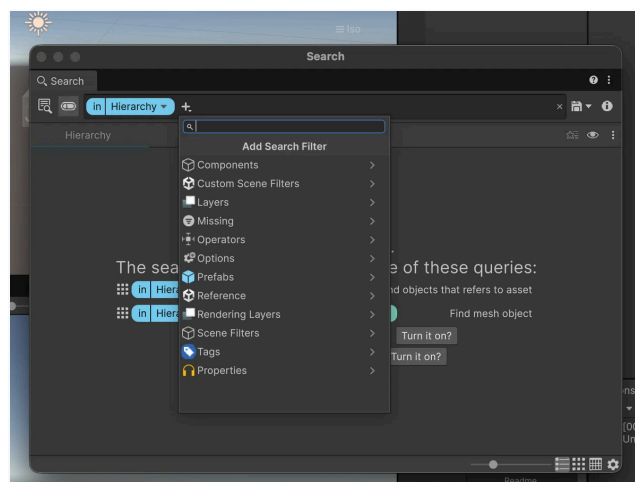
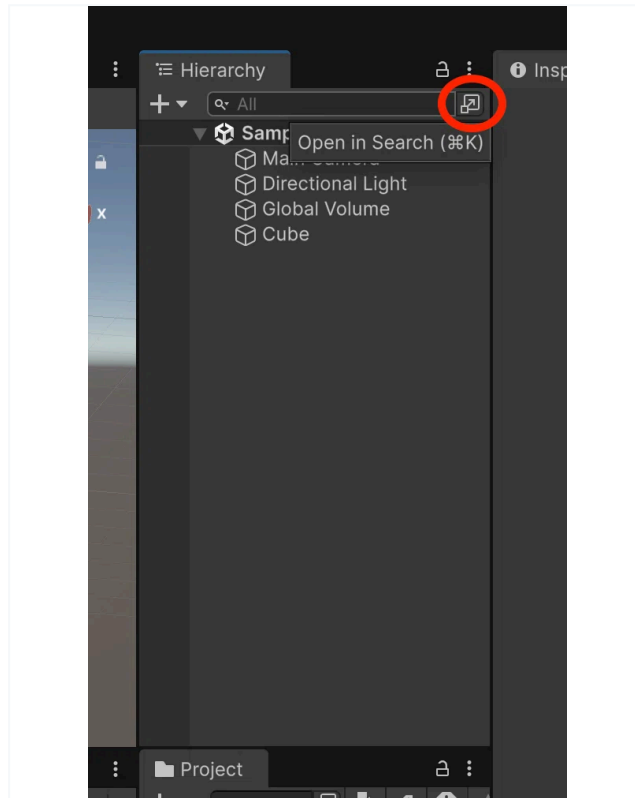


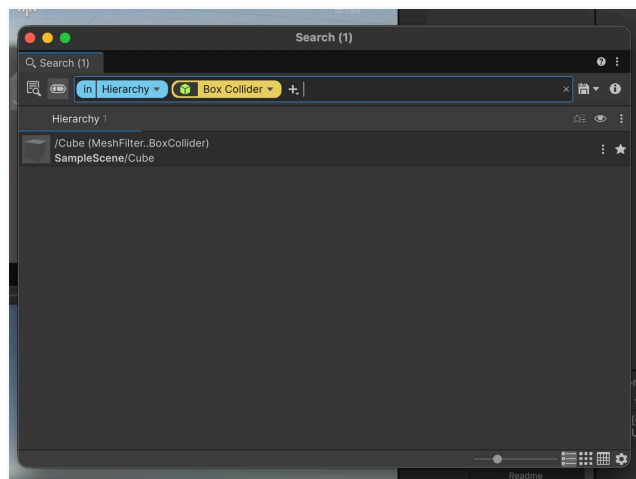
▶を押すと1個下の階層オブジェクトを開くことができますが、キャラクターのボーンとかだとかかなりの親子関係になってるかと思います。その時便利なのが「Altキー」



Altキー押ししながら  押すと全て開くことができます👍

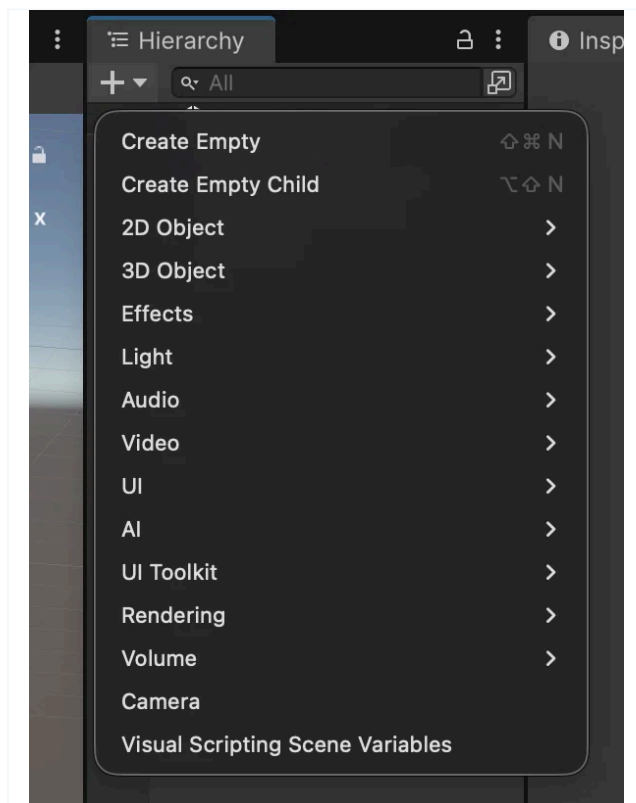
シーンの中のオブジェクトを検索できたりもする





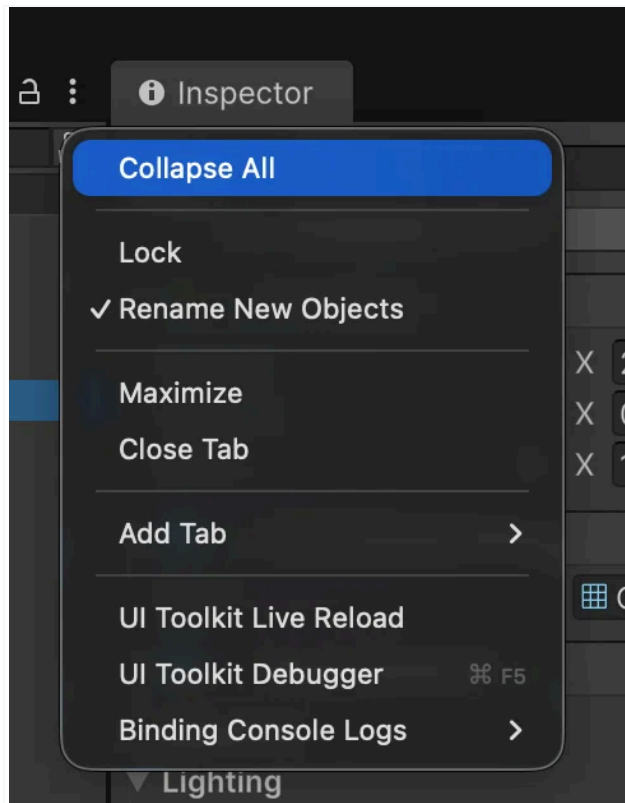
各コンポーネントがどのオブジェクトについているかなども確認できるため、例えば一律で削除したいコンポーネントをここで検索し削除することも可能。

オブジェクト生成



「+」からテンプレートをもとにオブジェクトを生成できる。
毎回空オブジェクト(EmptyObject)から作らなくてOK。

三点リーダーを押したら出てくるメニュー



Collapse All：全て折りたたむ。親子階層開いてても全て閉じます。
何でもかんでも開き過ぎた時に便利。

Lock：ビューの内容の固定。

Rename New Objects：新しいオブジェクト作った時にリネーム状態にするかどうか。
いちいちリネーム状態になるのが面倒！って方はoffっても良い。

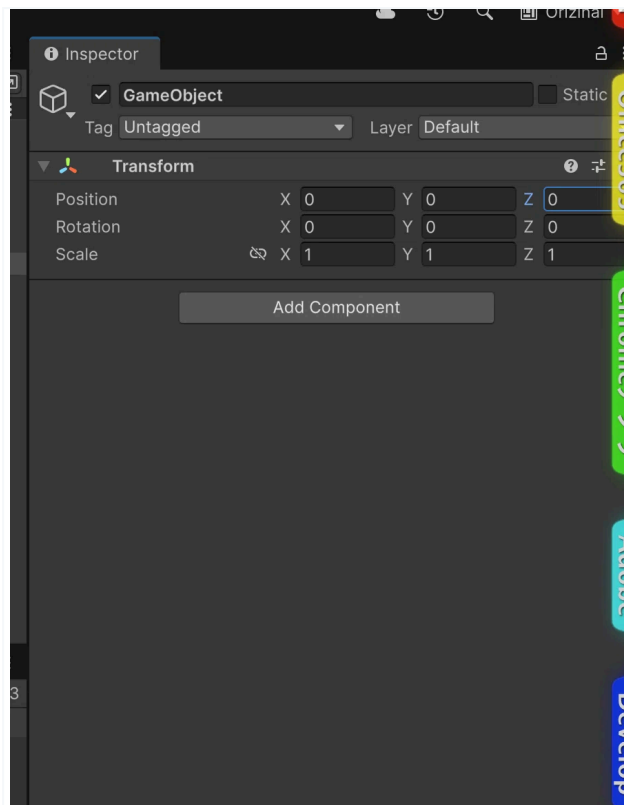
Maximize：最大化

Close Tab：タブ閉じる

Add Tab：ビューの追加

Inspectorビュー：オブジェクトの詳細/設定の編集

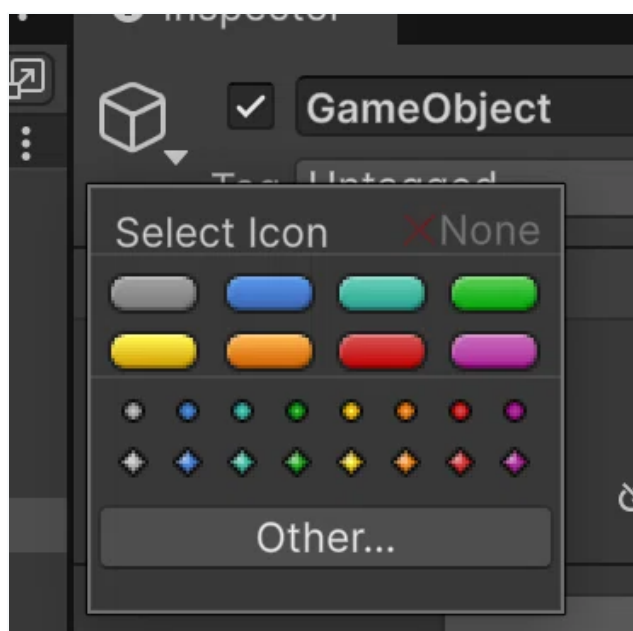
選択したオブジェクトの**詳細設定（コンポーネント）**を表示するのがInspectorビュー。
位置・角度・マテリアル・スクリプトなどをここで編集します。



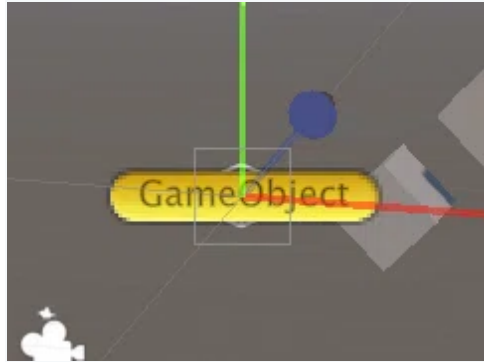
💡 ポイント

- 「Transform」で位置・回転・スケールを調整
- 「Add Component」で機能を追加
- チェックをつけることでオブジェクトのOn/Off
- Tag : Untaggedはタグなし。オブジェクトが地面か？プレイヤーか？などの種類の判定に便利。使いすぎは管理が難しくなるので注意。

Icon

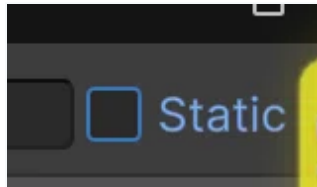


キューブアイコンをクリック
例えば黄色を押すと...？



Sceneビューにアイコン表示。
Sceneビューがわかりやすくなったりする。

Static



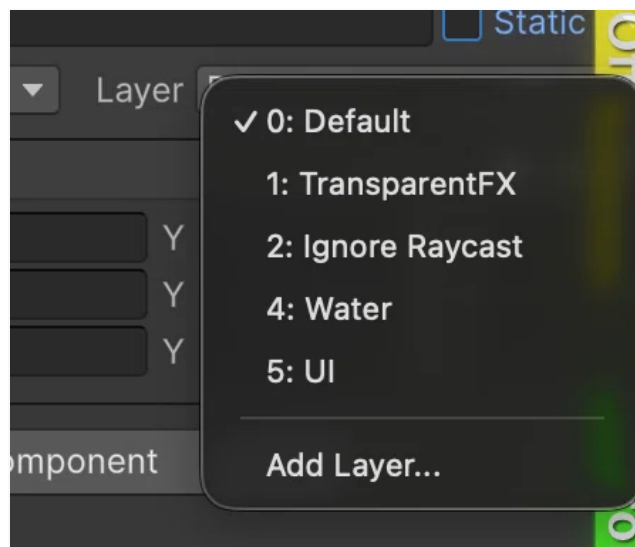
Static：静的。動かないオブジェクトに設定してあげる。

例) 地面、建物、壁...

これをOnにすることで処理を簡略化し最適化できる 🍷

Lightmapping/OcclusionCulling/BakedGI/ReflectionProbe/NavigationStatic
などで有効。

Layer



オブジェクトにレイヤーという所属を持たせる。
見えるものの指定/当たり方/照らされ方などを管理するタグのようなもの。

0:Default→デフォルト

1:TransparentFX→半透明エフェクト用。透過処理を最適化した描画順序が設定される。ただ、これは最近だと使わないかも。

2:Ignore Raycast→Raycast(光線)を無視。クリック判定を無視させたい時などに利用。

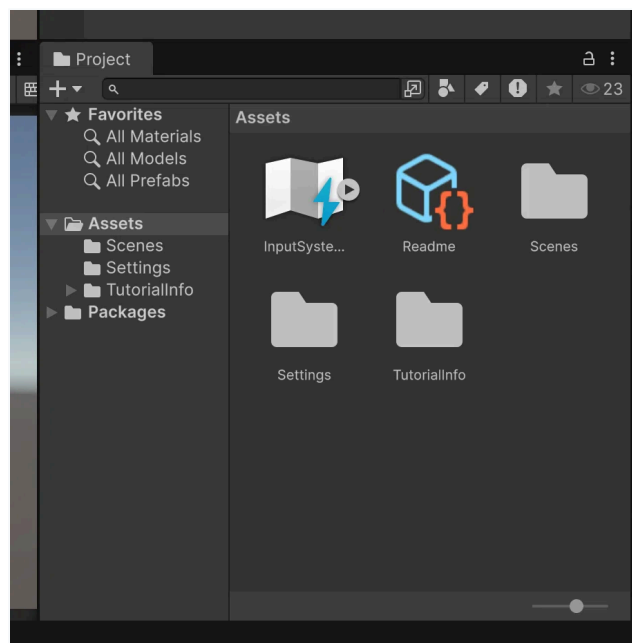
4:Water→水面用レイヤー。反射などを表現する際に使ったりする。

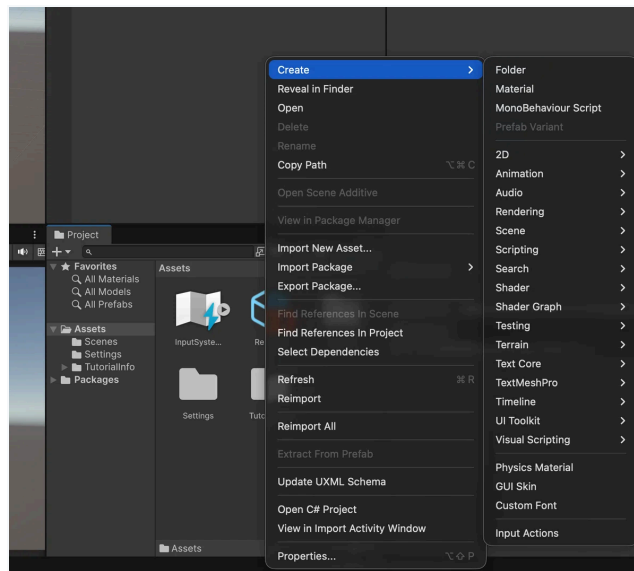
5:UI→CanvasやボタンなどのUI要素に使う。カメラの表示対象などを分けるのに便利。

Add Layer...→レイヤーを追加

Projectビュー:アセットを管理する倉庫

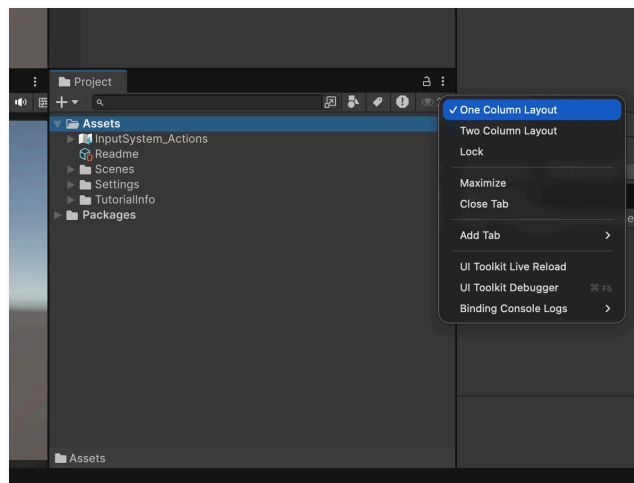
テクスチャ、スクリプト、Prefabなど**すべてのデータを管理する場所**です。実際のフォルダ構造と一致しており、Assetsフォルダ配下にあります。



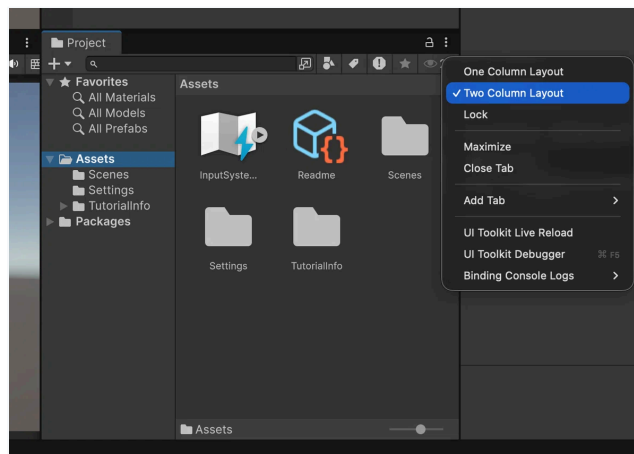


右クリックでファイル作成。これはよく使う。
また、左上の「+」からも同様にできる。

レイアウト



OneColumnLayout：1カラムのレイアウト



Two Column Layout：2カラムのレイアウト

ディレクトリ構成

プロジェクトでどういうフォルダを作るかなどを決める必要があります。

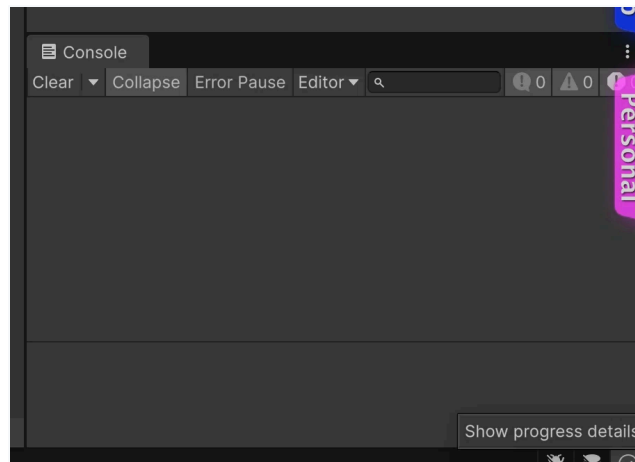
例)Scriptsフォルダ,Scenesフォルダ,Soundsフォルダ...

👉 こういうのを参考に

<https://unity.com/ja/how-to/organizing-your-project>

📺 Consoleビュー:エラーやログを確認する場所

スクリプトを動かしたときの**メッセージ・警告・エラー**が表示されるビュー。
ゲームをデバッグするために欠かせないビューです。



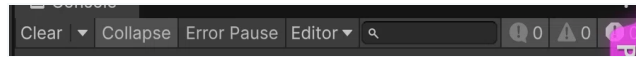
💡 ポイント

- 白い文字：ログ (Debug.Log)
- 黄色い文字：警告 (Debug.LogWarning)
- 赤い文字：エラー (Debug.LogError)
- メッセージをクリックすると、関連するスクリプトの行へジャンプできる
- 詳細には処理順やエラーメッセージなどもみれるため、常に確認しながらゲーム開発を進めるのが大事!! ✨

💬 Debug.Logの使い方

```
public class Ehehe : MonoBehaviour
{
    void Start()
    {
        Debug.Log("エヘヘ");
        Debug.LogWarning("エヘヘ");
    }
}
```

```
        Debug.LogError("エヘヘ");  
    }  
}
```



Clear : 表示されている"ログ"を全て消す

Collapse(コラプス) : 同じメッセージを1行にまとめて表示する。

Onにしておいた方が良い🔴

Updateメソッドで同じログがなん度も出たら他ログが追えなくなるからね。

Error Pause : エラーが出たら実行を一時停止(Pause)する。エラーの原因を探る時などに使い勝手が良い。

検索バー : ログ内を検索し、ヒットしたワードを含むメッセージのみ表示。

メッセージ・警告・エラーアイコン : 表示非表示の切り替えを行える。

最後に

これらを全て覚えなくてもゲームは作れますが、便利な機能ばかりです。

これ確認したいな...これしたいな...というときは

ビュー自体の機能でもしかしたらあるかもしれません。

探ってみましょうd(´`´)